BurjX Senior Mobile Engineer Assessment

Option 4: Multi-Step KYC Onboarding State Machine

Frontend-only Expo / React Native / TypeScript. Timebox: 90 minutes target, 2 hours maximum.

Option 4: Multi-Step KYC Onboarding State Machine

Assignment Summary

Build a frontend-only Expo React Native TypeScript KYC onboarding flow for a crypto exchange. The user should enter personal information, address details, and document details, then review and submit the application into a simulated verification process.

This assignment tests whether you can model a sensitive, multi-step, service-backed workflow without relying on a backend. All verification status changes, required-more-info states, and persistence behavior must be simulated locally in TypeScript.

What You Are Building

- Multi-step KYC wizard.
- Personal info, address, document, review, and status screens.
- Local draft recovery after app reload.
- Fake service-driven status updates.
- `requires_more_info`, approved, rejected, retry, and loading states.
- Clear state model for local draft versus fake service state.

Frontend Fake Service Contracts

```
export type KycStatus =
  | 'not_started'
  | 'draft'
  | 'submitted'
  | 'requires_more_info'
  | 'approved'
  | 'rejected';

export type KycStep =
  | 'personal_info'
  | 'address'
  | 'document'
  | 'review'
  | 'status';

export type KycRequiredField =
  | 'personalInfo.legalName'
  | 'personalInfo.dateOfBirth'
  | 'personalInfo.nationality'
  | 'address.country'
  | 'address.city'
  | 'address.line1'
  | 'document.type'
  | 'document.documentNumber';
```

```ts
export interface KycApplication {
  id: string;
  status: KycStatus;
  currentStep: KycStep;
  personalInfo?: {
    legalName: string;
    dateOfBirth: string;
    nationality: string;
  };
  address?: {
    country: string;
    city: string;
    line1: string;
  };
  document?: {
    type: 'passport' | 'national_id' | 'drivers_license';
    documentNumber: string;
  };
  rejectionReason?: string;
  requiredFields?: KycRequiredField[];
  updatedAt: string;
}
```

```ts
export async function fetchKycApplication(): Promise<KycApplication>;
export async function saveKycDraft(
  patch: Partial<KycApplication>
): Promise<KycApplication>;
export async function submitKycApplication(
  applicationId: string
): Promise<KycApplication>;
export async function pollKycStatus(): Promise<KycApplication>;
```

The fake service should deterministically support submitted, requires-more-info, approved, rejected, and failure states. Business outcomes are represented by `KycStatus`; service-like failures should be simulated by rejected promises. It should not use a backend or real identity/KYC provider.

Required Behavior

- Resume from last known step after app reload.
- Do not silently lose unsynced local changes.
- Explain and implement a conflict strategy when local draft and fake service state differ.
- Handle these source-of-truth cases:
- local draft is newer while fake service status is still `draft`
- fake service is `submitted`, `approved`, or `rejected`, so local edits must not overwrite it
- fake service returns `requires_more_info`, so route to the first relevant required field
- Validate required fields per step.
- Handle loading, saving, submitting, polling, retry, approved, rejected, and more-info states.
- Route `requires_more_info` back to the relevant step.
- Avoid logging sensitive KYC data.
- Store only what is needed locally and explain production encryption changes.
- Bound polling behavior so it does not run forever.

Required Tests

- Step validation.
- Allowed state transitions.
- Resume from local draft.
- Fake service state versus local draft conflict.
- `requires_more_info` routing.
- Approved and rejected outcomes.
- Fake service failure and retry.
- Polling cleanup or bounded retry behavior.

- No sensitive-data logging in intentional app code.

Evaluation Focus

- State machine/reducer design.
- Separation of UI, validation, storage, and fake service logic.
- Sensitivity of KYC data.
- Async state handling.
- Tests for transitions and validation.
- Communication of tradeoffs and AI usage.

Shared Candidate Rules

- Build from scratch using Expo, React Native, and TypeScript.
- Target 90 minutes. Maximum 2 hours.
- Build frontend-only. Do not create or require a backend, server, database, HTTP integration, websocket integration, Firebase/Supabase service, real exchange API, blockchain SDK, or secret keys.
- Any service behavior should be implemented as local TypeScript modules inside the app.
- AI tools are allowed, but candidates must explain what they asked AI, what they accepted or rejected, and how they verified the output.
- UI polish is secondary. Architecture, tests, security judgment, fake service design, and communication matter more.

Shared Fake Service Guidance

The service contracts in each assignment are not backend requirements. Implement them as local async TypeScript functions inside the app, for example in `src/services/fakeTradingService.ts` or similar.

Good fake services should:

- return `Promise` values and simulate latency with a small `delay` helper
- use deterministic inputs to trigger states, such as `twoFactorCode === '000000'` for invalid 2FA or old timestamps for stale quotes
- keep duplicate/idempotency checks in module state or a small in-memory map
- expose predictable failure cases that tests can cover
- avoid real network calls, API keys, or backend setup
- be easy to replace with real service adapters later

Example fake service pattern:

```
const delay = (ms = 250) => new Promise((resolve) => setTimeout(resolve, ms));

const seenRequests = new Map<string, unknown>();
type IdempotentInput = { clientRequestId?: string; clientOrderId?: string };

export async function fakeSubmit<TInput extends IdempotentInput, TResult>(
  input: TInput,
  buildResult: () => TResult
): Promise<TResult> {
  await delay();
  const key = input.clientRequestId ?? input.clientOrderId;

  if (!key) {
    throw new Error('MISSING_IDEMPOTENCY_KEY');
  }

  if (seenRequests.has(key)) {
    return seenRequests.get(key) as TResult;
  }

  if (key === 'network-error') {
    throw new Error('NETWORK_ERROR');
  }
```

```
  const result = buildResult();
  seenRequests.set(key, result);
  return result;
}
```

Final Deliverables

Submit the shared final deliverables listed at the end of this PDF. Your repo must include the KYC flow, local fake KYC service, typed models/contracts, tests for transitions/validation/resume behavior, and a README or notes file. Do not include a backend, server, HTTP API, real KYC provider, document upload service, or secret keys.

Shared Final Deliverables

Submit these at the end:

- A GitHub repository link containing the Expo React Native TypeScript project.
- If the repo is private, grant BurjX reviewer access before the deadline.
- Tests runnable with one command.
- A Google Drive or similar cloud-drive link to the full working-session screen recording with audio.
- Make sure the video link permissions allow BurjX reviewers to watch without requesting access.
- A `README.md` or `NOTES.md` in the repo covering setup instructions, how to run the app, how to run tests, architecture choices, testing strategy, AI usage summary, accepted/rejected AI output, security/reliability considerations, assumptions, tradeoffs, and what was intentionally left out due to the timebox.

The recording must show:

- how the task was understood and decomposed
- where AI was used and how output was verified
- at least one test being written or explained
- the core app behavior
- one self-review: what should be challenged before merging
- one product/security/production-service question to ask before shipping